23IT11E – WEBFRAMEWORKS USING PYTHON

Micro Project

Report On

Farmers Consultation Portal

Submitted by

MAYAVATHI.E 24205044

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

INFORMATION TECHNOLOGY



NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

OCTOBER – 2026

NATIONAL ENGINEERING COLLEGE

K.R.NAGAR, KOVILPATTI – 628503

BONAFIDE CERTIFICATE

This is to certify that this project report is a Bonafide record of **MAYAVATHI.E (24205044)** work done for the course **23IT11E – WEBFRAMEWORKS USING PYTHON** at the NATIONAL ENGINEERING COLLEGE, K.R. NAGAR, during the academic year 2026 who carried out the project work under my supervision.

Course Instructor/Guide

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	4
1	INTRODUCTION	5
2	IMPLEMENTATION	7
3	IMPLEMENTATION RESULTS	51
4	CONCLUSION	54

ABSTRACT

The GATE CSE Preparation Portal is a web application that organizes core study resources for students preparing for the Graduate Aptitude Test in Engineering (Computer Science and Information Technology). The portal brings course-wise learning materials and administrator-managed links into one simple interface.

The application provides student registration and login, a separate administrator account choice on the shared login page, eight GATE CSE course groups, and an administrator dashboard for creating, editing and deleting website or video resources. Students can open a course and browse its available materials. Account and resource data are stored in MongoDB; passwords are stored as Werkzeug password hashes.

The project is implemented with Python and Flask, Jinja templates, HTML and CSS, and the PyMongo driver. Its purpose is to demonstrate a clear CRUD-based web application with authentication, role checks, MongoDB persistence and a responsive course-oriented user experience. It provides a foundation for future additions such as progress tracking, mock tests and study planning.

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

The GATE examination requires sustained preparation across several computer science subjects. Students commonly collect notes, videos and reference websites from different sources. When these resources are scattered across browser bookmarks and files, it becomes harder to find a useful item again or keep materials organized by subject.

The GATE CSE Preparation Portal addresses this problem with a web-based catalog of eight course groups. A student signs in, selects a course and opens its listed resources. An administrator maintains the shared catalog through create, read, update and delete operations. The application uses Flask routes to handle requests, Jinja templates to render pages and MongoDB collections to persist users and materials.

1.2 PROBLEM STATEMENT

Students need a simple way to locate preparation resources by GATE CSE subject. A manually maintained collection can become inconsistent, difficult to update and unavailable to other students. A web portal with course grouping and an administrator-managed resource list provides a central place to discover and maintain those learning links.

1.3 OBJECTIVES

- Provide one portal for eight core GATE CSE course groups.
- Allow students to register, log in and browse course materials.
- Provide an administrator role that can manage study resources.
- Store user and resource records in MongoDB.
- Demonstrate CRUD operations, sessions, password hashing, form validation and role-based route protection using Flask.

1.4 SCOPE

The current project covers student accounts, administrator access, course browsing and CRUD management of website and video links. It does not currently implement mock examinations, automatic online question collection, PDF upload, study-plan scheduling or analytics. These are possible extensions and are not represented as existing features in this report.

1.5 EIGHT GATE CSE COURSE GROUPS

- Engineering Mathematics
- Digital Logic
- Computer Organization and Architecture
- Programming and Data Structures
- Algorithms

- Theory of Computation and Compiler Design
- Operating Systems
- Databases and Computer Networks

CHAPTER 2
IMPLEMENTATION

2.1 TECHNOLOGIES USED

• Python 3 and Flask: application routes, sessions, request processing and template rendering.

• MongoDB and PyMongo: persistent storage and database queries.

• Werkzeug security utilities: password hashing and password verification.

• HTML, CSS and Jinja: page structure, styling and dynamic rendering.

• Microsoft Edge: local browser used to run and capture the portal screens.

2.2 SYSTEM REQUIREMENTS

The project requires Python, the packages listed in requirements.txt, a running MongoDB service and a modern web browser. MongoDB connection and database names can be configured with MONGO_URI and MONGO_DB. The default local database name in the application is gate_prep.

2.3 PROJECT STRUCTURE

• app.py — Flask application, database connection, route handlers and CRUD logic.

• templates/ — Jinja HTML pages for the landing page, login, registration, courses, course resources, administrator dashboard and resource form.

• static/ — shared CSS and other static assets.

• requirements.txt — Python package dependencies.

2.4 DATABASE DESIGN

The application uses the MongoDB database named gate_prep by default. The users collection stores a name, normalized email, password_hash and role. A unique index on email prevents duplicate accounts. The materials collection stores title, subject, description, URL and resource_type. The resource type is Website or Video. MongoDB ObjectId values identify resource records and are used by edit and delete routes.

2.5 APPLICATION WORKFLOW

• A visitor opens the landing page and chooses to log in or register as a student.

• A student account is created with a hashed password and a user role; successful login creates a Flask session.

• The student opens the course list and selects one of the eight course groups.

• The course page retrieves matching materials from MongoDB and displays their links.

• An administrator selects Admin on the login form and signs in with an administrator account.

• The administrator dashboard lists resources and provides actions to add, edit or delete a material.

• Protected routes check the session role before returning student or administrator pages.

2.6 CRUD OPERATIONS

Create: the administrator submits a resource form; validated values are inserted into MongoDB. Read: the dashboard lists stored resources and each course page retrieves resources for its subject. Update: the administrator opens an existing resource, changes its fields and saves the updated values. Delete: the administrator submits the delete action for a selected ObjectId. This set of operations demonstrates persistent CRUD functionality through Flask routes and PyMongo.

2.7 AUTHENTICATION AND ACCESS CONTROL

Student and administrator accounts use a shared login page with an account-type selector. Passwords are hashed with Werkzeug before storage and checked during login. Flask session values hold the signed-in user identifier, display name and role. A route decorator requires a valid session and, where specified, the required role. Registration is limited to student accounts.

2.8 VALIDATION AND ERROR HANDLING

The registration flow checks a minimum password length and handles duplicate email addresses. The resource form checks that title, description and URL are present, that the course belongs to the supported list and that the resource type is valid. Invalid identifiers and unknown course slugs return a not-found response; unauthorized roles are rejected.

CHAPTER 3
IMPLEMENTATION RESULTS

The following screenshots were captured from the project running locally with its MongoDB connection. They show the pages available in the current implementation.

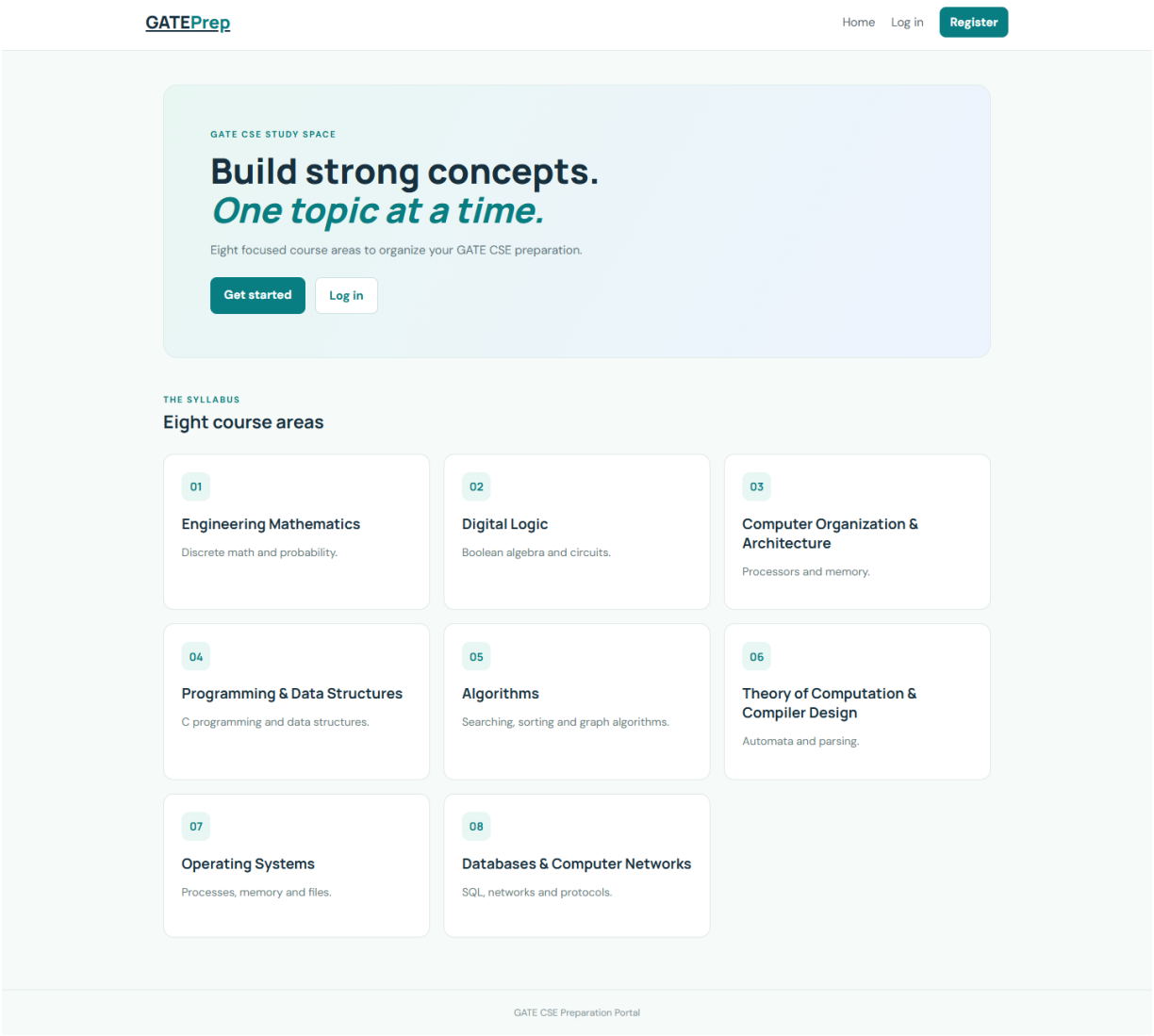


Figure 3.1: Portal landing page

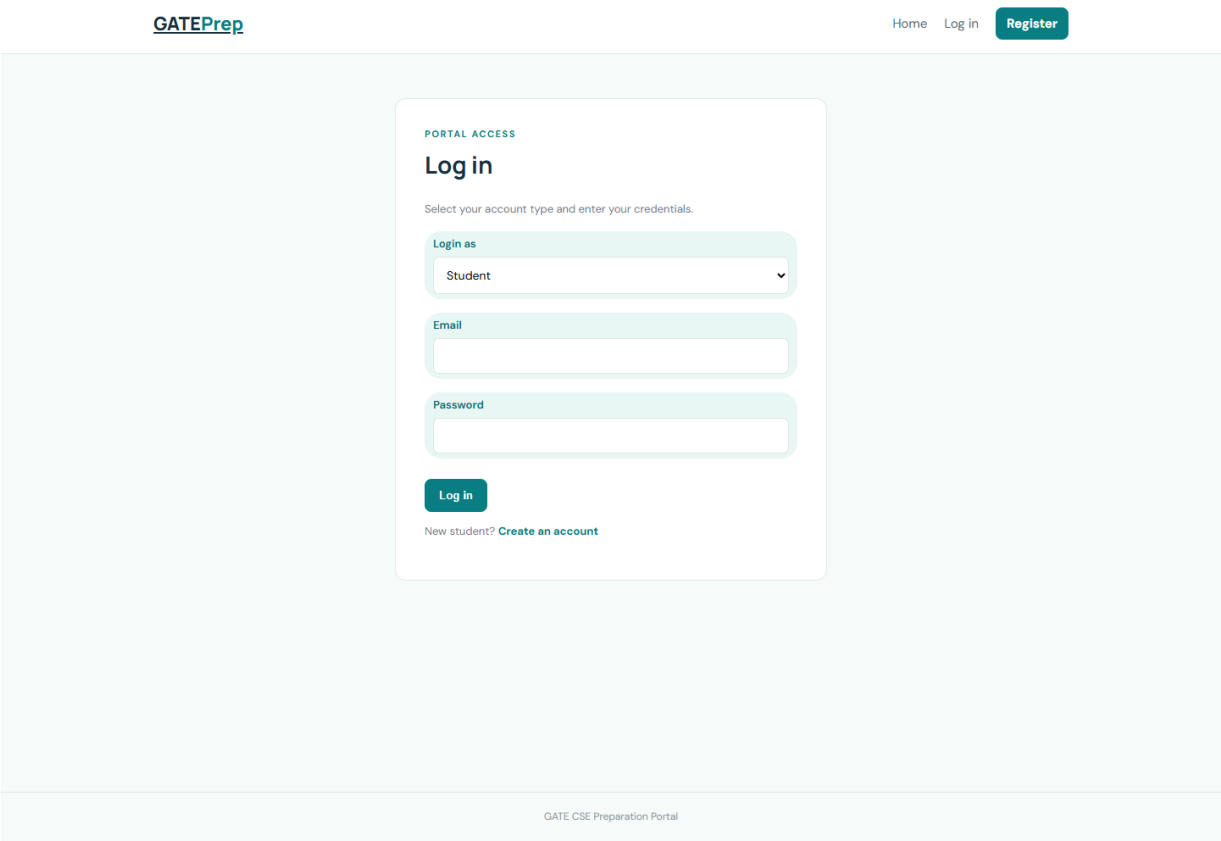


Figure 3.2: Shared login page with student and administrator selection

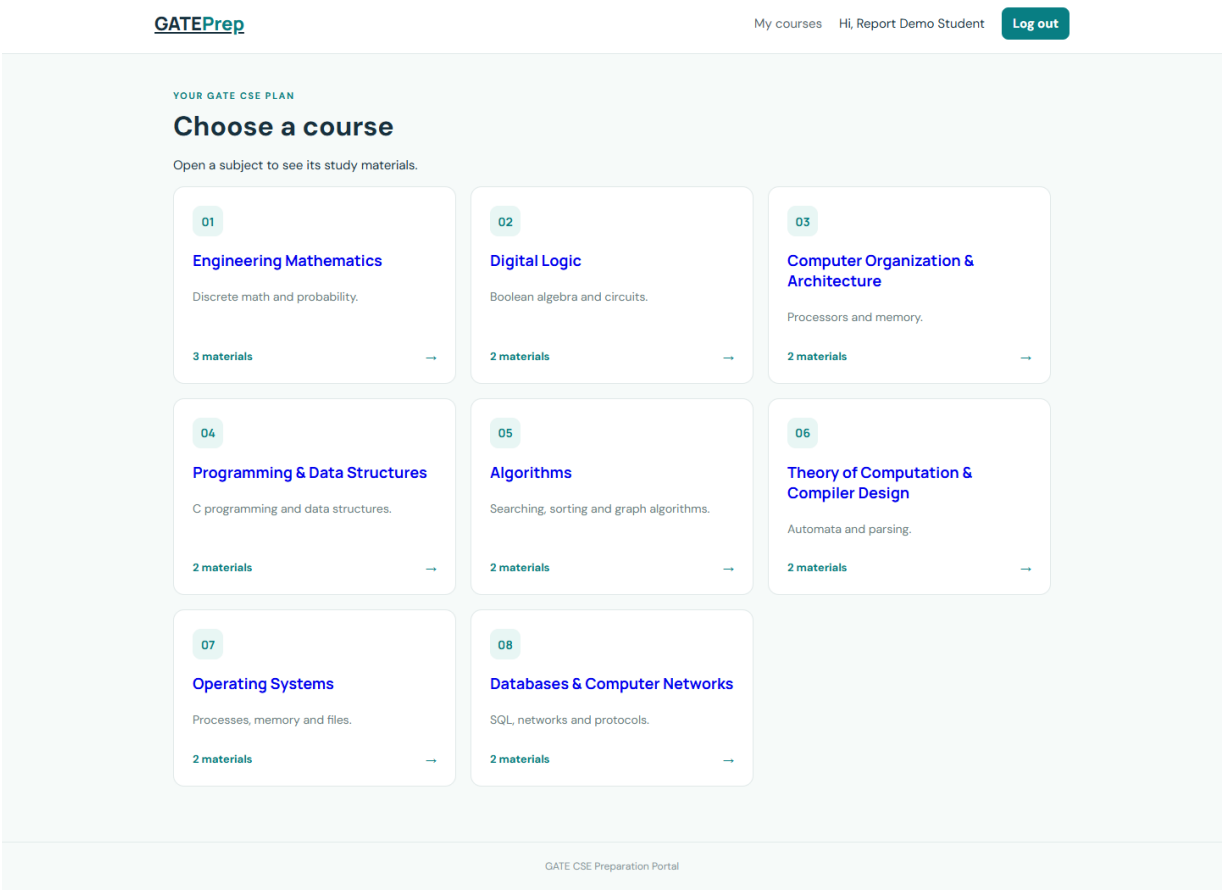


Figure 3.3: Student view of the eight GATE CSE course groups

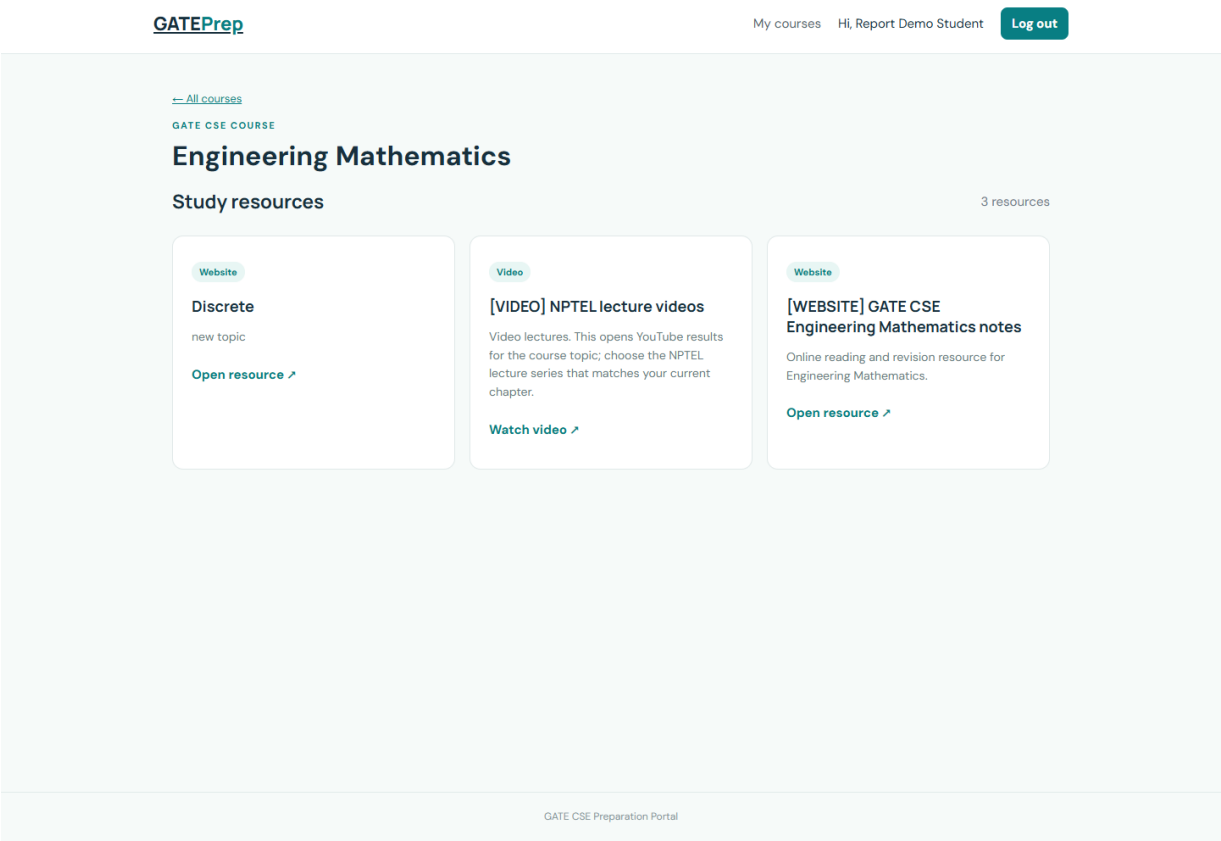


Figure 3.4: Course resource listing

GATEPrep

Admin dashboard Hi, Administrator Log out

PORTAL MANAGEMENT

Admin dashboard

Manage the resources shown in each course.

+ Add resource

Databases & Computer Networks · Website

[WEBSITE] NPTEL Database Management Systems course

Online reading and revision resource for Databases & Computer Networks.

Open resource ↗
Edit Delete

Databases & Computer Networks · Video

[VIDEO] NPTEL lecture videos

Video lectures. This opens YouTube results for the course topic; choose the NPTEL lecture series that matches your current chapter.

Open resource ↗
Edit Delete

Operating Systems · Website

[WEBSITE] Operating Systems GATE previous-year questions

Online reading and revision resource for Operating Systems.

Open resource ↗
Edit Delete

Operating Systems · Video

[VIDEO] NPTEL lecture videos

Video lectures. This opens YouTube results for the course topic; choose the NPTEL lecture series that matches your current chapter.

Open resource ↗
Edit Delete

Theory of Computation & Compiler Design · Website

[WEBSITE] Theory of Computation tutorial

Online reading and revision resource for Theory of Computation & Compiler Design.

Open resource ↗
Edit Delete

Theory of Computation & Compiler Design · Video

[VIDEO] NPTEL lecture videos

Video lectures. This opens YouTube results for the course topic; choose the NPTEL lecture series that matches your current chapter.

Open resource ↗
Edit Delete

Algorithms · Website

[WEBSITE] GATE Algorithms previous-year practice

Online reading and revision resource for Algorithms.

Open resource ↗
Edit Delete

Algorithms · Video

[VIDEO] NPTEL lecture videos

Video lectures. This opens YouTube results for the course topic; choose the NPTEL lecture series that matches your current chapter.

Open resource ↗
Edit Delete

Programming & Data Structures · Website

[WEBSITE] DSA guide for GATE CS

Online reading and revision resource for Programming & Data Structures.

Open resource ↗
Edit Delete

Programming & Data Structures · Video

[VIDEO] NPTEL lecture videos

Video lectures. This opens YouTube results for the course topic; choose the NPTEL lecture series that matches your current chapter.

Open resource ↗
Edit Delete

Computer Organization & Architecture · Website

[WEBSITE] COA for GATE study guide

Online reading and revision resource for Computer Organization & Architecture.

Open resource ↗
Edit Delete

Computer Organization & Architecture · Video

[VIDEO] NPTEL lecture videos

Video lectures. This opens YouTube results for the course topic; choose the NPTEL lecture series that matches your current chapter.

Open resource ↗
Edit Delete

Digital Logic · Website

[WEBSITE] Digital Logic and gates guide

Online reading and revision resource for Digital Logic.

Open resource ↗
Edit Delete

Digital Logic · Video

[VIDEO] NPTEL lecture videos

Video lectures. This opens YouTube results for the course topic; choose the NPTEL lecture series that matches your current chapter.

Open resource ↗
Edit Delete

Engineering Mathematics · Website

[WEBSITE] GATE CSE Engineering Mathematics notes

Online reading and revision resource for Engineering Mathematics.

Open resource ↗
Edit Delete

Engineering Mathematics · Video

[VIDEO] NPTEL lecture videos

Video lectures. This opens YouTube results for the course topic; choose the NPTEL lecture series that matches your current chapter.

Open resource ↗
Edit Delete

Engineering Mathematics · Website

Discrete

new topic

Open resource ↗
Edit Delete

Figure 3.5: Administrator dashboard for resource management

3.1 RESULT SUMMARY

The portal starts locally, connects to MongoDB and serves the landing page. Student registration and login lead to the course area, where the eight subject groups are available. Administrator login opens the resource dashboard. Resources can be created, listed, edited and deleted, and course pages show matching resource records. The screenshots document these visible interfaces.

CHAPTER 4
CONCLUSION

4.1 SUMMARY OF THE PROJECT

The GATE CSE Preparation Portal was developed to provide students with a single place to find learning links arranged by subject. The portal brings together eight course groups: Engineering Mathematics, Digital Logic, Computer Organization and Architecture, Programming and Data Structures, Algorithms, Theory of Computation and Compiler Design, Operating Systems, and Databases and Computer Networks. This organization makes it easier for students to identify the materials relevant to the topic they are studying.

4.2 FEATURES COMPLETED

The student workflow supports account registration, login, course browsing and viewing course-specific resources. The administrator workflow supports adding, viewing, editing and deleting website or video resources. These are the four basic CRUD operations. A resource added by an administrator is stored in MongoDB and can then be displayed to students under its selected course. Updates and deletions are reflected in later database reads, keeping the course pages aligned with the maintained resource list.

4.3 CONCEPTS DEMONSTRATED

The project demonstrates how a Flask application connects URL routes to Python functions and uses templates to render dynamic HTML pages. Forms pass user input to the server, where the application validates values before saving them. PyMongo connects the application to MongoDB collections and performs inserts, queries, updates and deletes. Password hashing avoids storing plain-text passwords, while Flask sessions remember a signed-in user. Role checks protect administrator functions from student accounts. Together, these concepts show how a small web application can be divided into clear request, validation, storage and presentation steps.

4.4 CURRENT SCOPE AND LIMITATIONS

The current implementation focuses on account access, course-wise study links and administrator-managed resource records. The resource types currently supported are website and video links. It does not automatically collect questions or materials from the internet, conduct mock examinations, upload PDF files, generate personalized study schedules, or calculate student performance analytics. Stating this scope clearly distinguishes working features from proposed extensions and provides a practical baseline for future development.

4.5 FUTURE ENHANCEMENTS

The portal can be extended with structured notes and administrator-uploaded PDF materials, provided file type and size are validated and storage is handled safely. A study planner could let students set subject goals and mark planned topics as complete. Progress tracking could record completed resources and revision status. Mock tests could use a curated and reviewed question bank, store attempts, and show scores by subject or topic. Revision reminders and performance summaries could then help students decide which topics to revisit. Each enhancement should have

its own data model, validation rules and user interface so that it integrates cleanly with the existing CRUD workflows.

4.6 FINAL REMARKS

The completed project provides a useful foundation for a GATE CSE preparation portal and a clear demonstration of Flask and MongoDB in a student-level web application. Its central resource catalog addresses the basic need to organize study links, while separate student and administrator roles give each user an appropriate set of actions. The application can be run and extended incrementally: first by adding verified learning content, and later by introducing planning, revision and assessment features. The project therefore meets its present CRUD-focused objective and leaves a structured path for further improvement.

REFERENCES

Flask Documentation, https://flask.palletsprojects.com/

MongoDB Manual, https://www.mongodb.com/docs/manual/

PyMongo Documentation, https://www.mongodb.com/docs/languages/python/pymongo-driver/current/

Werkzeug Security Helpers, https://werkzeug.palletsprojects.com/en/stable/utils/

APPENDIX A — FLASK APPLICATION SOURCE (app.py)

Source file: app.py

```
import os
from functools import wraps
from flask import Flask, session, request, redirect, url_for, render_template, flash, abort
from werkzeug.security import generate_password_hash, check_password_hash
from pymongo import MongoClient, ASCENDING
from pymongo.errors import DuplicateKeyError
from bson import ObjectId
app = Flask(__name__); app.secret_key = os.environ.get("SECRET_KEY", "change-this-secret")
client = MongoClient(os.environ.get("MONGO_URI", "mongodb://127.0.0.1:27017"), serverSelectionTimeoutMS=3000);
client.admin.command("ping")
mongo = client[os.environ.get("MONGO_DB", "gate_prep")]
# Eight GATE CSE course groups
COURSES = [
("engineering-mathematics","Engineering Mathematics","Discrete math and probability."), ("digital-logic","Digital
Logic","Boolean algebra and circuits."),
("computer-organization","Computer Organization & Architecture","Processors and memory."), ("programming-data-
structures","Programming & Data Structures","C programming and data structures."),
("algorithms","Algorithms","Searching, sorting and graph algorithms."), ("toc-compiler-design","Theory of Computation &
Compiler Design","Automata and parsing."),
("operating-systems","Operating Systems","Processes, memory and files."), ("dbms-networks","Databases & Computer
Networks","SQL, networks and protocols.")]
TYPES = {"website":"Website", "video":"Video"}
mongo.users.create_index("email", unique=True)
admin_email = os.environ.get("ADMIN_EMAIL", "admin@gate.local").lower()
if not mongo.users.find_one({"email": admin_email}):

mongo.users.insert_one({"name":"Administrator","email":admin_email,"password_hash":generate_password_hash(os.environ.get
("ADMIN_PASSWORD","admin123")),"role":"admin"})
def records(items): return [dict(item, id=str(item["_id"])) for item in items]
def need(role=None):
    def decorate(view):
        @wraps(view)
        def check(*args, **kwargs):
            if not session.get("user_id"): return redirect(url_for("login"))
            if role and session.get("role") != role: abort(403)
            return view(*args, **kwargs)
        return check
    return decorate
@app.route("/")
def index():
    if session.get("role"): return redirect(url_for("admin" if session["role"]=="admin" else "courses"))
    return render_template("index.html", courses=COURSES)
@app.route("/register", methods=["GET","POST"])
def register():
    if request.method == "POST":
        name=request.form["name"].strip(); email=request.form["email"].strip().lower(); pw=request.form["password"]
        if len(pw)<6: flash("Password needs at least 6 characters.","error")
        else:
            try:
                user={"name":name,"email":email,"password_hash":generate_password_hash(pw),"role":"user"}
                user_id=mongo.users.insert_one(user).inserted_id
                session.update(user_id=str(user_id),name=name,role="user")
                return redirect(url_for("courses"))
            except DuplicateKeyError: flash("Email already registered.","error")
    return render_template("register.html")
@app.route("/login", methods=["GET","POST"])
def login():
    if request.method == "POST":
        role={"admin":"admin","student":"user"}.get(request.form["account_type"])
        user=mongo.users.find_one({"email":request.form["email"].strip().lower()})
        if role and user and user["role"]==role and check_password_hash(user["password_hash"],request.form["password"]):
            session.clear(); session.update(user_id=str(user["_id"]),name=user["name"],role=role)
            return redirect(url_for("admin" if role=="admin" else "courses"))
        flash("Check account type, email, and password.","error")
    return render_template("login.html")
@app.route("/logout")
def logout(): session.clear(); return redirect(url_for("index"))
@app.route("/courses")
@need("user")
def courses():
    counts={c[1]:mongo.materials.count_documents({"subject":c[1]}) for c in COURSES}
    return render_template("courses.html",courses=COURSES,counts=counts)
@app.route("/courses/<slug>")
@need("user")
```

```
def course(slug):
    selected=next((c for c in COURSES if c[0]==slug),None)
    if not selected: abort(404)
    items=records(mongo.materials.find({"subject":selected[1]}).sort("title",1))
    return render_template("course.html",course=selected,materials=items,resource_types=TYPES)
@app.route("/admin")
@need("admin")
def admin(): return render_template("admin.html",materials=records(mongo.materials.find().sort("_id",-
1)),resource_types=TYPES)
@app.route("/admin/material/new",methods=["GET","POST"])
@app.route("/admin/material/<item_id>/edit",methods=["GET","POST"])
@need("admin")
def resource_form(item_id=None):
    item=mongo.materials.find_one({"_id":ObjectId(item_id)}) if item_id and ObjectId.is_valid(item_id) else None
    item_id and not item and abort(404)
    if request.method=="POST":
        title=request.form["title"].strip(); subject=request.form["subject"]
        desc=request.form["description"].strip(); url=request.form["url"].strip(); kind=request.form["resource_type"]
        if not title or not desc or not url or subject not in [c[1] for c in COURSES] or kind not in TYPES:
            flash("Complete all fields and choose a valid course and type.","error")
        else:
            values={"title":title,"subject":subject,"description":desc,"url":url,"resource_type":kind}
            mongo.materials.update_one({"_id":item["_id"]},{"$set":values}) if item else
mongo.materials.insert_one(values)
            return redirect(url_for("admin"))
    item=records([item])[0] if item else None
    return render_template("material_form.html",material=item,courses=COURSES,resource_types=TYPES)
@app.post("/admin/material/<item_id>/delete")
@need("admin")
def delete_resource(item_id):
    if not ObjectId.is_valid(item_id): abort(404)
    mongo.materials.delete_one({"_id":ObjectId(item_id)})
    return redirect(url_for("admin"))
if __name__=="__main__":
    app.run(debug=True)
```

APPENDIX B — HTML TEMPLATE: admin.html

Source file: templates/admin.html

```
{% extends "base.html" %}{% block title %}Admin dashboard{% endblock %}{% block content %}<div class="row"><div><div
class="kicker">PORTAL MANAGEMENT</div><h1>Admin dashboard</h1><p>Manage the resources shown in each course.</p></div><a
class="btn" href="/admin/material/new">+ Add resource</a></div><section class="grid">{% for m in materials %}<article
class="card"><label>{{m.subject}}</label><span class="resource-
badge">{{resource_types.get(m.resource_type,'Website')}}</span><h3>{{m.title}}</h3><p>{{m.description}}</p><a
href="{{m.url}}" target="_blank" rel="noopener">Open resource ↗</a><div class="actions"><a
href="/admin/material/{{m.id}}/edit">Edit</a><form method="post" action="/admin/material/{{m.id}}/delete"
onsubmit="return confirm('Delete this resource?')"><button class="link danger">Delete</button></form></div></article>{%
else %}<div class="empty">No resources yet. Add a website or video link.</div>{% endfor %}</section>{% endblock %}
```

APPENDIX C — HTML TEMPLATE: base.html

Source file: templates/base.html

```
<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-
scale=1"><title>{% block title %}GATE Prep{% endblock %} | GATE CSE</title><link rel="stylesheet"
href="{{url_for('static',filename='style.css')}}"></head><body><header><a class="brand"
href="/">GATE<span>Prep</span></a><nav>{% if session.get('role')=='user' %}<a href="/courses">My courses</a>{% elif
session.get('role')=='admin' %}<a href="/admin">Admin dashboard</a>{% else %}<a href="/">Home</a><a href="/login">Log
in</a>{% endif %}{% if session.get('user_id') %}<span>Hi, {{session.get('name')}}</span><a class="btn small"
href="/logout">Log out</a>{% else %}<a class="btn small" href="/register">Register</a>{% endif %}</nav></header><main>{%
with ms=get_flashed_messages(with_categories=true) %}{% for c,m in ms %}<div class="flash {{c}}">{{m}}</div>{% endfor
%}{% endwith %}{% block content %}{% endblock %}</main><footer>GATE CSE Preparation Portal</footer></body></html>
```

APPENDIX D — HTML TEMPLATE: course.html

Source file: templates/course.html

```
{% extends "base.html" %}{% block title %}{{course[1]}}{% endblock %}{% block content %}<a class="back-link"
href="/courses">← All courses</a><div class="kicker">GATE CSE COURSE</div><h1 class="page-title">{{course[1]}}</h1><div
class="row"><h2>Study resources</h2><span>{{materials|length}} resources</span></div><section class="grid">{% for m in
materials %}<article class="card"><span class="resource-
badge">{{resource_types.get(m.resource_type,'Website')}}</span><h3>{{m.title}}</h3><p>{{m.description}}</p><a
class="resource-action" href="{{m.url}}" target="_blank" rel="noopener noreferrer">{% if m.resource_type=='video'
%}Watch video ↗{% else %}Open resource ↗{% endif %}</a></article>{% else %}<div class="empty">No resources for this
course yet.</div>{% endfor %}</section>{% endblock %}
```

APPENDIX E — HTML TEMPLATE: courses.html

Source file: templates/courses.html

```
{% extends "base.html" %}{% block title %}Your courses{% endblock %}{% block content %}<div class="kicker">YOUR GATE CSE
PLAN</div><h1 class="page-title">Choose a course</h1><p class="muted">Open a subject to see its study
materials.</p><section class="grid">{% for c in courses %}<a class="card course-card course-link"
href="{{url_for('course',slug=c[0])}}"><span
class="number">{{"%02d"|format(loop.index)}}</span><h3>{{c[1]}}</h3><p>{{c[2]}}</p><span class="course-
count">{{counts.get(c[1],0)}} materials <b>→</b></span></a>{% endfor %}</section>{% endblock %}
```

APPENDIX F — HTML TEMPLATE: index.html

Source file: templates/index.html

```
{% extends "base.html" %}{% block title %}GATE CSE Prep{% endblock %}{% block content %}<section class="hero"><div
class="kicker">GATE CSE STUDY SPACE</div><h1>Build strong concepts.<br><em>One topic at a time.</em></h1><p>Eight
focused course areas to organize your GATE CSE preparation.</p><div class="hero-actions"><a class="btn"
href="/register">Get started</a><a class="btn secondary" href="/login">Log in</a></div></section><div
class="row"><div><div class="kicker">THE SYLLABUS</div><h2>Eight course areas</h2></div></div><section class="grid">{%
for c in courses %}<article class="card course-card"><span
class="number">{{"%02d"|format(loop.index)}}</span><h3>{{c[1]}}</h3><p>{{c[2]}}</p></article>{% endfor %}</section>{%
endblock %}
```

APPENDIX G — HTML TEMPLATE: login.html

Source file: templates/login.html

```
{% extends "base.html" %}
{% block title %}Login{% endblock %}
{% block content %}
<form class="form card" method="post">
  <div class="kicker">PORTAL ACCESS</div>
  <h1>Log in</h1>
  <p class="muted">Select your account type and enter your credentials.</p>


  <label>Login as
    <select name="account_type" required>
      <option value="student">Student</option>
      <option value="admin">Admin</option>
    </select>
  </label>
  <label>Email<input name="email" type="email" required autocomplete="username"></label>
  <label>Password<input name="password" type="password" required autocomplete="current-password"></label>
  <button class="btn">Log in</button>
  <p>New student? <a href="/register">Create an account</a></p>
</form>
{% endblock %}
```

APPENDIX H — HTML TEMPLATE: material_form.html

Source file: templates/material_form.html

```
{% extends "base.html" %}{% block title %}Resource{% endblock %}{% block content %}<form class="form card"
method="post"><a class="back-link" href="/admin">← Admin dashboard</a><h1>{{'Edit' if material else 'Add'}}
resource</h1><label>Title<input name="title" required value="{{material.title if material else
''}}"></label><label>Course<select name="subject" required><option value="">Choose a course</option>{% for c in courses
%}<option value="{{c[1]}}" {% if material and material.subject==c[1] %}selected{% endif %}>{{c[1]}}</option>{% endfor
%}</select></label><label>Description<textarea name="description" rows="4" required>{{material.description if material
else ''}}</textarea></label><label>Resource type<select name="resource_type" required>{% for key,label in
resource_types.items() %}<option value="{{key}}" {% if material and material.resource_type==key %}selected{% endif
%}>{{label}}</option>{% endfor %}</select></label><label>Resource URL<input name="url" type="url" required
value="{{material.url if material else ''}}" placeholder="https://..."></label><button class="btn">Save
resource</button></form>{% endblock %}
```

APPENDIX I — HTML TEMPLATE: register.html

Source file: templates/register.html

```
{% extends "base.html" %}{% block title %}Student registration{% endblock %}{% block content %}<form class="form card"
method="post"><div class="kicker">STUDENT ACCOUNT</div><h1>Create account</h1><label>Name<input name="name"
required></label><label>Email<input name="email" type="email" required></label><label>Password (6+ characters)<input
name="password" type="password" minlength="6" required></label><button class="btn">Register and start</button><p>Already
registered? <a href="/login/student">Student login</a></p></form>{% endblock %}
```